



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

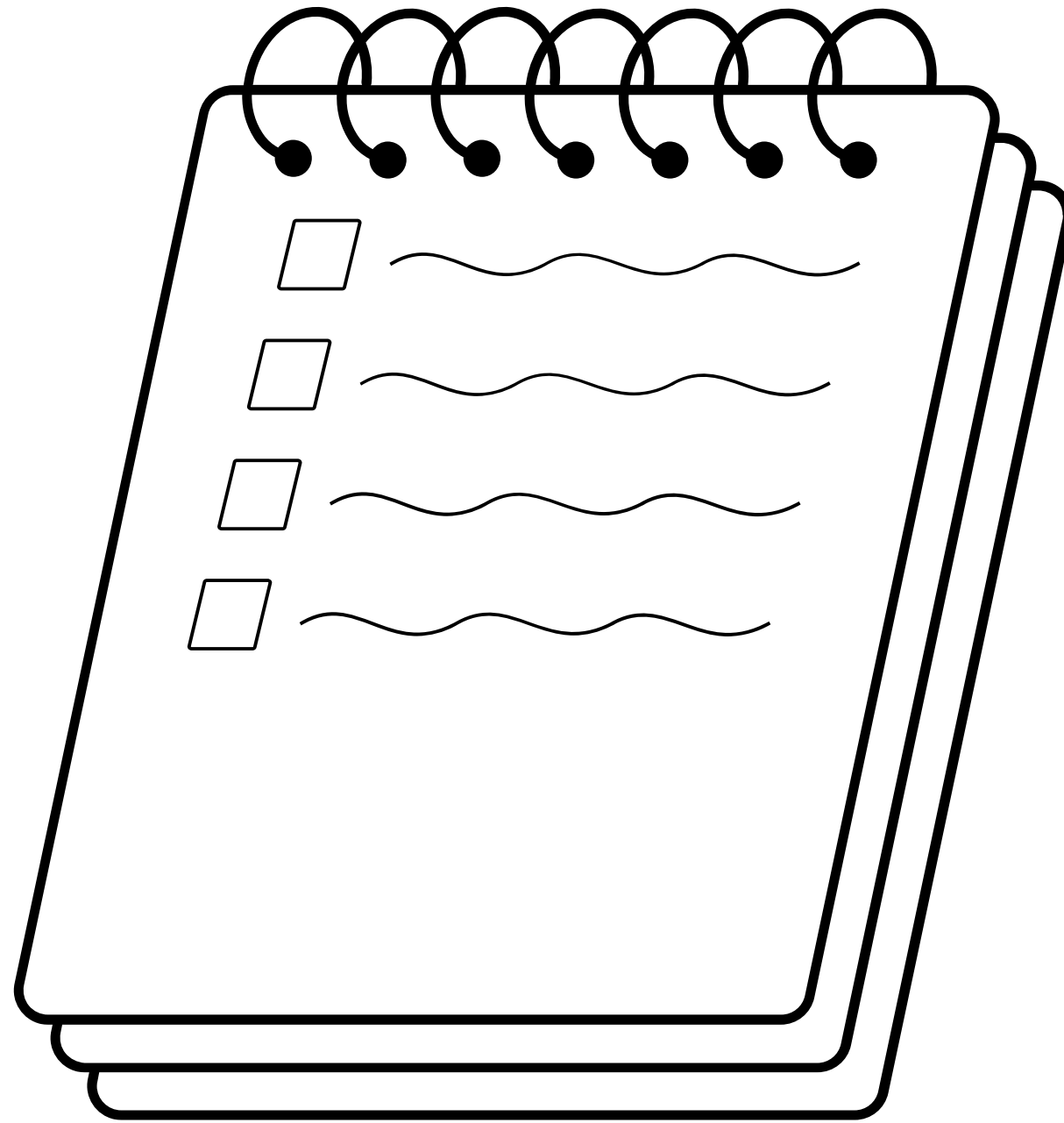
Weather Forecasting and Monitoring System

Big Data Storage and Processing

Group 13

<i>Phan Trần Việt Bách</i>	<i>20225435</i>
<i>Nguyễn Trí Thanh</i>	<i>20225457</i>
<i>Lê Nhật Quang</i>	<i>20225522</i>
<i>Nguyễn Hoàng Sơn</i>	<i>20225525</i>
<i>Nguyễn Minh Quân</i>	<i>20225520</i>

ONE LOVE. ONE FUTURE.



- 1. Introduction**
- 2. Data Source**
- 3. System Architecture**
- 4. Implementation Details**
- 5. Lessons Learned**
- 6. Conclusion**
- 7. Member Contributions**

1. Introduction

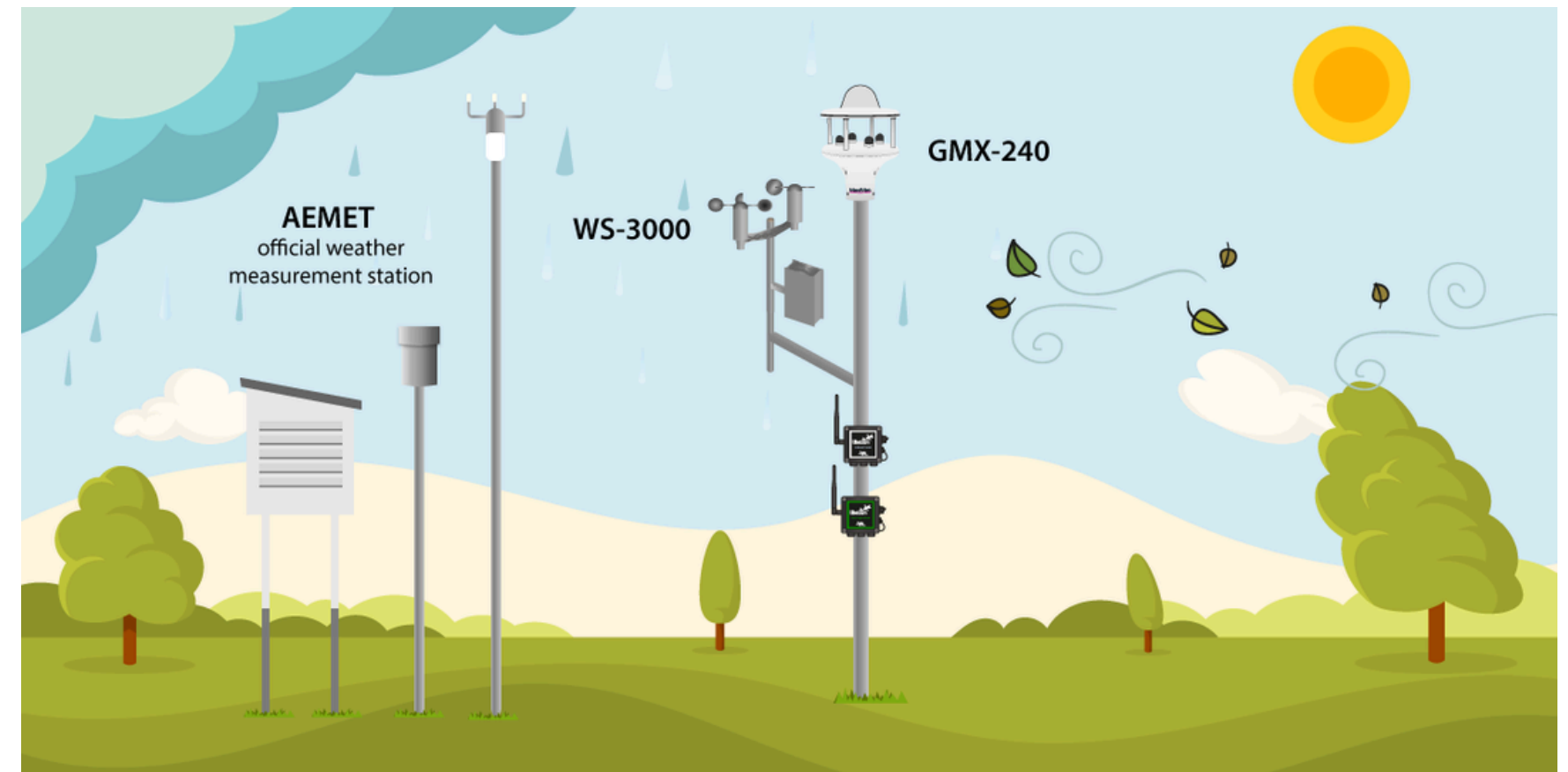
1. Introduction

- IoT-based weather stations generating large volumn of meteorological data

→ **Sensor noise, transmission errors, etc.**

- Raw environmental data is **not suitable for predictive modelling**
- Traditional data pipelines struggle to support:
 - **Real-time weather monitoring**
 - **High-quality** historical data for analysis

→ Utilize the **Lambda Architecture** to address both requirements with the **Speed** and **Batch** layers



2. Dataset

2.1. Data Source

Open-source Open-Meteo weather API

- No need for API key, daily cap of 10,000 daily API calls
- **Offers Python sample codes** for API setting up and data processing
- **Comprehensive past records** of weather conditions from many weather stations
- **Various temporal resolution:** daily, hourly, quarterly
- Data dates back to 1940

open-meteo/**open-meteo**



Free Weather Forecast API for non-commercial use

62
Contributors

1
Used by

243
Discussions

4k
Stars

279
Forks



2.2. Data Description

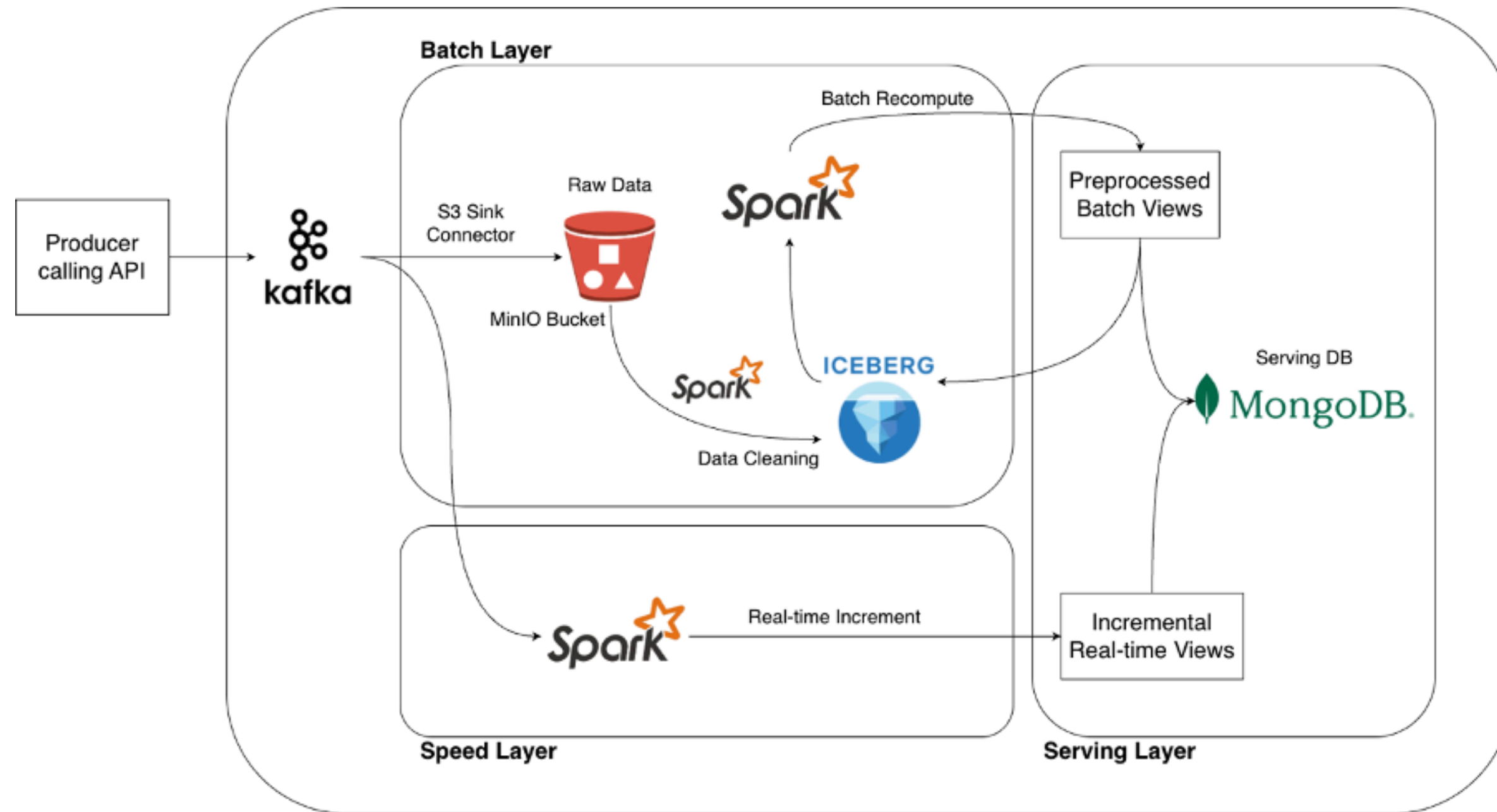
To balance between temporal resolution and data volume, our data crawled is:

- Hourly
- From 01/01/2020 to 31/12/2025
- 11 weather features:
 - **Thermodynamic:** temperature, relative humidity, dew point
 - **Dynamics and Radiative conditions:** wind speed and direction, cloud coverage, and sunshine duration
 - **Weather indicators:** precipitation, soil moisture and temperature, WMO weather code

```
1 {
2   "latitude": 52.52,
3   "longitude": 13.419,
4   "elevation": 44.812,
5   "generationtime_ms": 2.2119,
6   "utc_offset_seconds": 0,
7   "timezone": "Europe/Berlin",
8   "timezone_abbreviation": "CEST",
9   "hourly": {
10     "time": ["2022-07-01T00:00", "2022-07-01T01:00", "2022-07-01T02:00", ...],
11     "temperature_2m": [13, 12.7, 12.7, 12.5, 12.5, 12.8, 13, 12.9, 13.3, ...]
12   },
13   "hourly_units": {
14     "temperature_2m": "°C"
15   }
16 }
```

3. System Architecture

3. System Architecture



Architecture Overview

4. Implementation Details

4.1. Data Ingestion

- A producer app that calls Open-Meteo API
- Use Kafka Python client to produce messages
- Data at a location (lat, lon) within a time range

```
def fetch_weather(lat, lon, start_date, end_date):  
    url = (  
        f"{HISTORICAL_URL}?latitude={lat}&longitude={lon}&start_date={start_date}&end_date={end_date}"  
        "&hourly=temperature_2m,relative_humidity_2m,dew_point_2m,precipitation,"  
        "cloud_cover,sunshine_duration,wind_speed_10m,wind_direction_10m,weather_code,"  
        "soil_temperature_0_to_7cm,soil_moisture_0_to_7cm&timezone=UTC"  
    )  
    try:  
        response = requests.get(url, timeout=60)  
        response.raise_for_status()  
        return response.json()  
    except Exception as e:  
        print(f"Failed to fetch data: {e}")  
        return None
```

4.1. Data Ingestion

- Strimzi operator for Kafka on Kubernetes
- Use a dual-role node



```
apiVersion: kafka.strimzi.io/v1
kind: KafkaNodePool
# used for specifying pool of Kafka pods with roles
# replicas: num of pods with this config
# roles: broker only, controller only, both for dual-role
# labels: strimzi.io/cluster: <kafka-resource-name>: pods will follow Kafka resource configs under this name
metadata:
  name: dual-role
  namespace: bigdata-pipeline
  labels:
    strimzi.io/cluster: kafka-cluster
spec:
  replicas: 1
  roles:
    - controller
    - broker
  storage:
    type: jbod
    volumes:
      - id: 0
        type: persistent-claim
        size: 1Gi
        deleteClaim: false
        kraftMetadata: shared
```

4.1. Data Ingestion

- Expose 3 ports for internal without/with TLS and external
- Topic: openmeteo-data
- 3 partitions
- Use location name (city/province) as key for partitioning

```
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: openmeteo-data
  namespace: bigdata-pipeline
  labels:
    strimzi.io/cluster: kafka-cluster # metadata.name of kafka cluster
spec:
  partitions: 3
  replicas: 1 # adjust based on num. of brokers, for small-scale demo use 1
  config:
    retention.ms: 7200000 # time that events are stored by brokers; after this duration, clear
    segment.bytes: 52428800 # segmentation size of topic data; here, 500Mi
```

```
apiVersion: kafka.strimzi.io/v1
kind: Kafka
metadata:
  # a KafkaNodePool use `name` to get configs for Kafka nodes it create
  # Kafka nodes will be created with this name
  # specify for topics manifests to create such topics in nodes with given name
  name: kafka-cluster
  namespace: bigdata-pipeline
spec:
  kafka:
    version: 4.1.1
    metadataVersion: 4.1-IV1
    listeners:
      - name: plain
        port: 9092
        type: internal
        tls: false
      - name: tls
        port: 9093
        type: internal
        tls: true
      - name: external
        port: 9094
        type: loadbalancer
        tls: false
    config:
      offsets.topic.replication.factor: 1
      transaction.state.log.replication.factor: 1
      transaction.state.log.min.isr: 1
      default.replication.factor: 1
      min.insync.replicas: 1
  entityOperator:
    topicOperator: {}
    userOperator: {}
```

4.1. Data Ingestion

UI for Apache Kafka

83b5a60 v0.7.2

A

Dashboard

kafka-cluster

Brokers

Topics

Consumers

Topics

+ Add a Topic

Search by Topic Name

Show Internal Topics

Delete selected topics

Copy selected topic

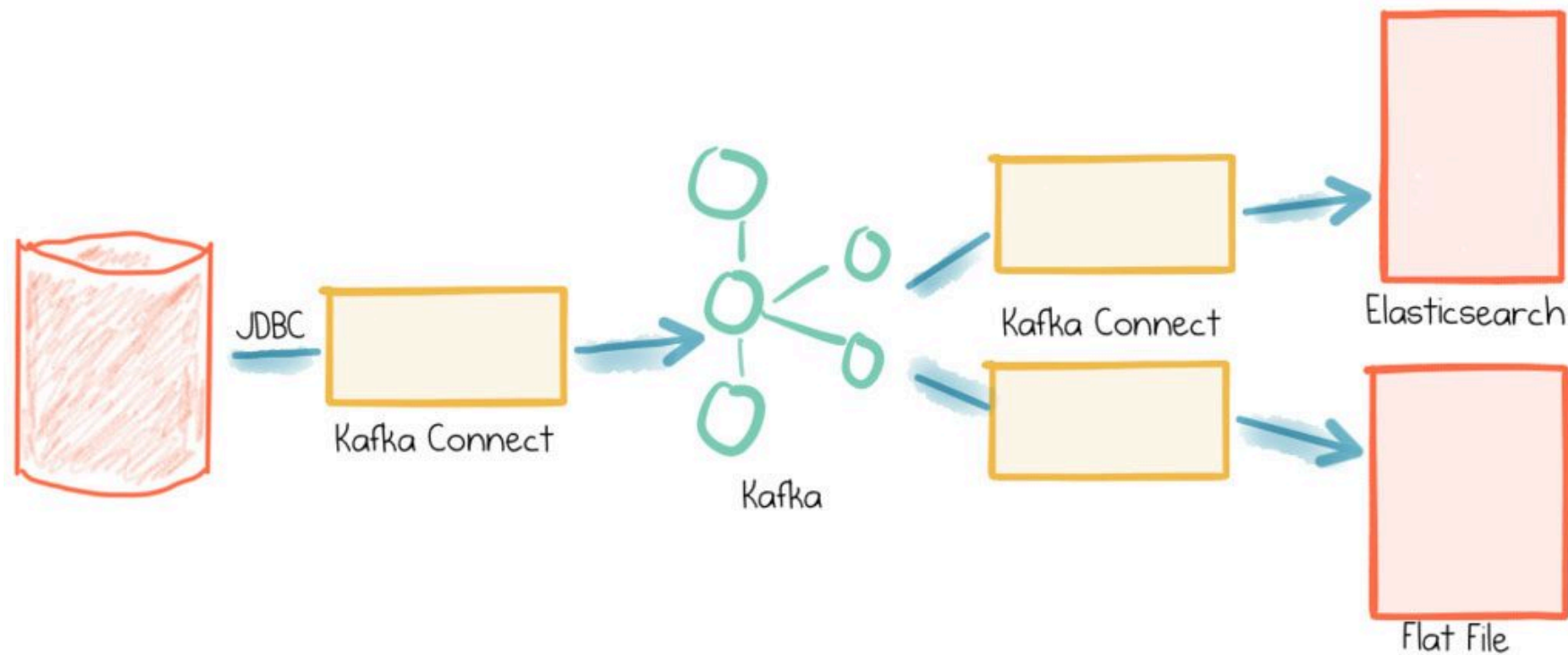
Purge messages of selected topics

☐	Topic Name	Partitions	Out of sync replicas	Replication Factor	Number of messages	Size	
☐	connect-configs	1	0	1	8	5 KB	
☐	connect-offsets	25	0	1	0	0 Bytes	
☐	connect-status	5	0	1	36	7 KB	
☐	openmeteo-data	3	0	1	0	0 Bytes	



4.1. Data Ingestion

- Kafka Connect for streamlining messages into storage
- Convenient way to move data between Kafka and data sources/sinks
- S3 Sink Connector to use with MinIO
- flush_size=100, rotate_interval=5 min



```
apiVersion: kafka.strimzi.io/v1
kind: KafkaConnector

metadata:
  name: minio-sink-connector
  namespace: bigdata-pipeline
  labels:
    strimzi.io/cluster: s3-sink-connect-cluster

spec:
  class: io.confluent.connect.s3.S3SinkConnector
  tasksMax: 3
  config:
    topics: openmeteo-data

    s3.region: "us-east-1"
    s3.bucket.name: raw-zone
    s3.endpoint: http://minio-internal-service:9000
    s3.path.style.access: true
    s3.force.path.style: true
    store.url: http://minio-internal-service:9000
    path.style.access.enabled: true

    # s3 credentials
    aws.access.key.id: minioadmin
    aws.secret.access.key: minioadmin123

    storage.class: io.confluent.connect.s3.storage.S3Storage
    format.class: io.confluent.connect.s3.format.json.JsonFormat

    key.converter: org.apache.kafka.connect.storage.StringConverter
    key.converter.schemas.enable: false
    value.converter: org.apache.kafka.connect.json.JsonConverter
    value.converter.schemas.enable: false

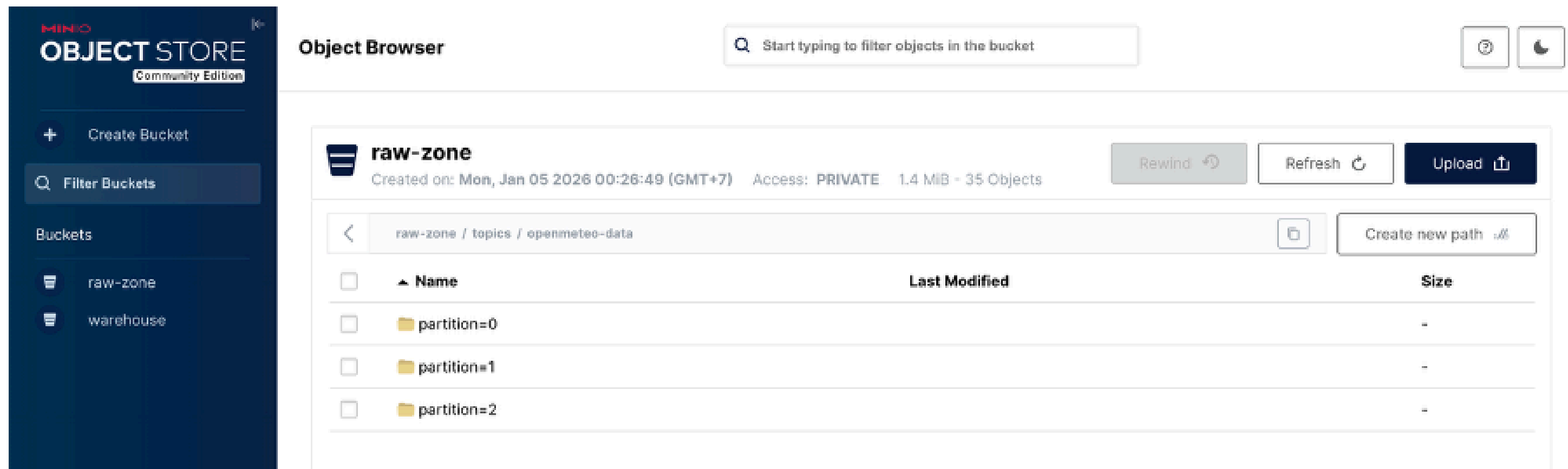
    flush.size: 100
    rotate.interval.ms: 300000

  schemaCompatibility: NONE
```


4.2. Data Storage

- MinIO: S3-compatible object storage
- → Capable for scaling up to using AWS S3 buckets
- Port 9000 for API access, used for data reads/writes
- Port 9001 for web console → LoadBalancer to access from local machine
- Use 1GB PVC for storage

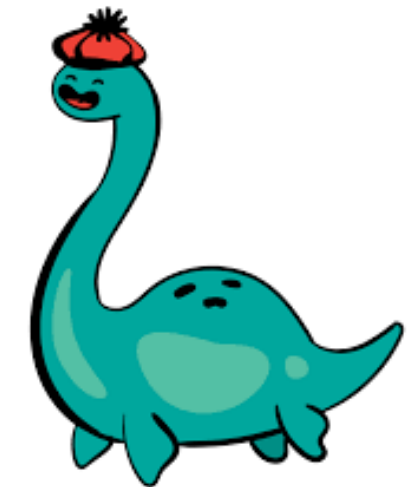
MINIO



4.2. Data Storage

- Apache Iceberg for table format
- Catalog: Nessie
- Centralized table metadata access
- Git-like version control
- Use MongoDB for version store

ICEBERG



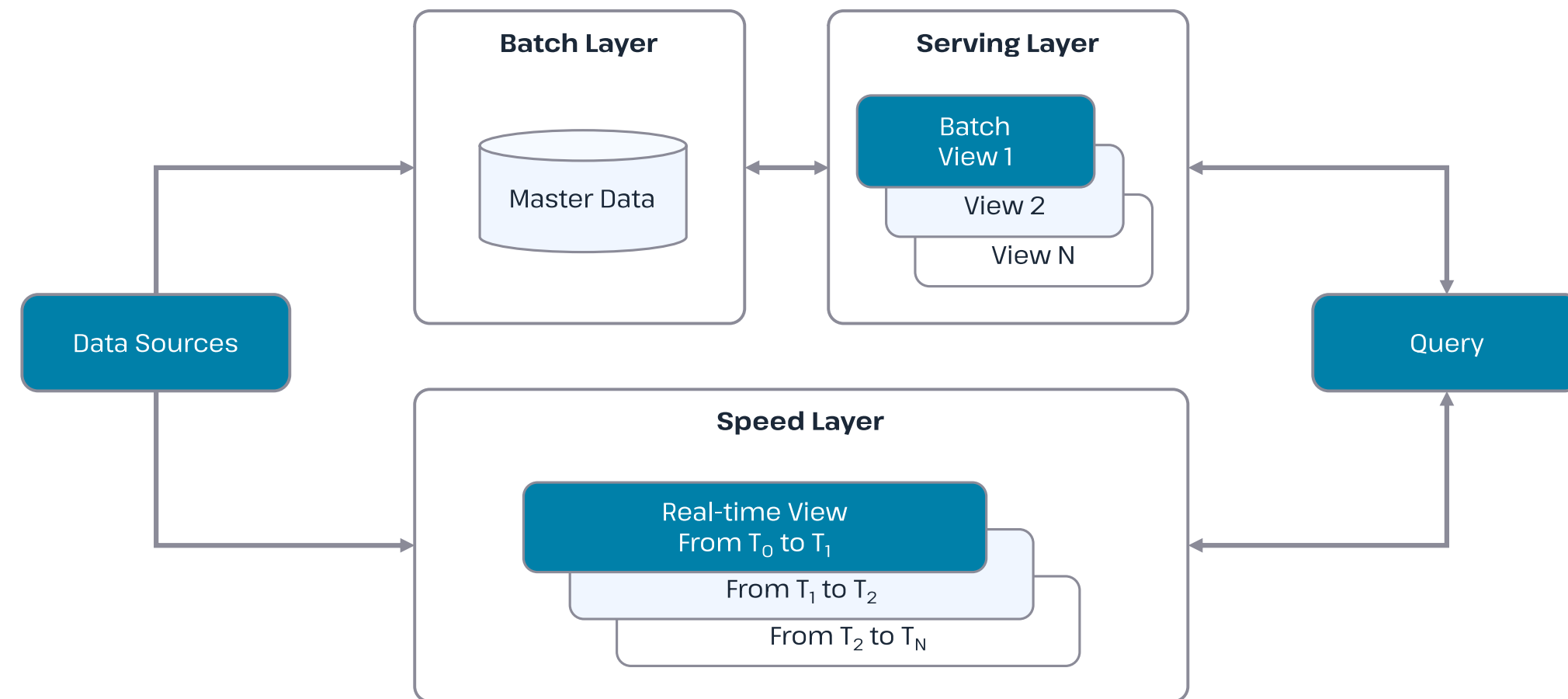
Nessie Tables



4.3. Spark Processing

The **Lambda Architecture** is characterized by its two parallel processing paths to balance low latency and high accuracy

- **Batch layer:**
 - Perform large-scale, offline, and scheduled computations to maximize accuracy
 - Medallion Architecture
- **Speed layer:**
 - Process data in real-time to minimize latency
 - Implemented with Kafka and Spark Structured Streaming



4.3.1. Batch Layer: Bronze-to-Silver Transformation

- Ensures consistent, physically valid, and continuous time-series data
- Divided into **four main stages**
- **Stage 1: Deduplication**
 - Imposes strict time-series integrity of **one observation/hour/city**
 - Composite keys: **[timestamp, city]**
- **Stage 2: Type Conversion, String Standardization, and Outlier Nullification**
 - **Type conversion:** Converts timestamp columns to their correct types
 - **String standardization:** Hà Nam-Nam Định (tây) → ha_nam_nam_dinh_tay
 - **Outlier Nullification:** Numerical weather data is validated against **known scientific bounds**
 - **Vector Decomposition for Wind Direction:**

$$\theta_{rad} = \theta_{deg} \times \frac{\pi}{180} \text{ where } \text{sine_component} = \sin(\theta_{rad}) \text{ and } \text{cosine_component} = \cos(\theta_{rad})$$

4.3.1. Batch Layer: Bronze-to-Silver Transformation

- **Stage 3: Imputation**

- A hybrid two-stage strategy with a fallback mechanism
- **Primary method: Weighted Moving Average (WMA)**
- **Fallback:** Forward-Fill → Backward-Fill → Default placeholder

$$\text{Value} = \frac{\sum (\text{Neighbor Value} \times \text{Weight})}{\sum \text{Existing Weights}}$$

- **Stage 4: Finalization**

- **Vector Recomposition:**

$$\theta_{\text{final_rad}} = \text{atan2}(\text{cosine_component}, \text{sine_component})$$

$$\theta_{\text{final_deg}} = \theta_{\text{final_rad}} \times \frac{180}{\pi}$$

$$\theta_{\text{norm}} = (\theta_{\text{final_deg}} + 360) \bmod(360)$$

- **Cleanup:** Drops temporary vector columns

4.3.1. Batch Layer: Silver-to-Gold Transformation

- Enriches the already cleaned Silver data with advanced weather features
- The data transition includes **four key steps**
- **Step 1: Cyclical Time Encoding**

$$x_{\sin} = \sin\left(2\pi \cdot \frac{t}{\text{period}}\right)$$

$$x_{\cos} = \cos\left(2\pi \cdot \frac{t}{\text{period}}\right)$$

- **Hour of Day:** Encoded as **hour_sin** and **hour_cos**
- **Day of Year:** Encoded as **day_of_year_sin** and **day_of_year_cos**
- **Step 2: Autoregressive (Lag) Features:**
 - Relies on the **Persistent Model** assumption
 - **Lagging features:** temperature and dew point temperature
 - **Lagging intervals:** 1 hour, 2 hours, 3 hours, and 24 hours

4.3.1. Batch Layer: Silver-to-Gold Transformation

- **Step 3: Climate Trend Discovery**

- **Rolling features:** temperature, dew point temperature, relative humidity, wind speed, and precipitation
- **Rolling statistics:** mean and standard deviation
- **Rolling windows:** 12 hours, 1 day, 3 days, and a week

- **Step 4: Advanced Meteorological Features**

- **Vapor Pressure Deficit:** The difference between how much moisture the air can hold and how much it actually holds → **The "drying power" of the air**

$$e_s = 6.112 \times \exp \left(\frac{17.67 \times T_{air}}{T_{air} + 243.5} \right) (hPa) \quad e_a = 6.112 \times \exp \left(\frac{17.67 \times T_{dew}}{T_{dew} + 243.5} \right) (hPa)$$

- **Apparent Temperature (AT):** A combined index to estimate how temperature feels to a human

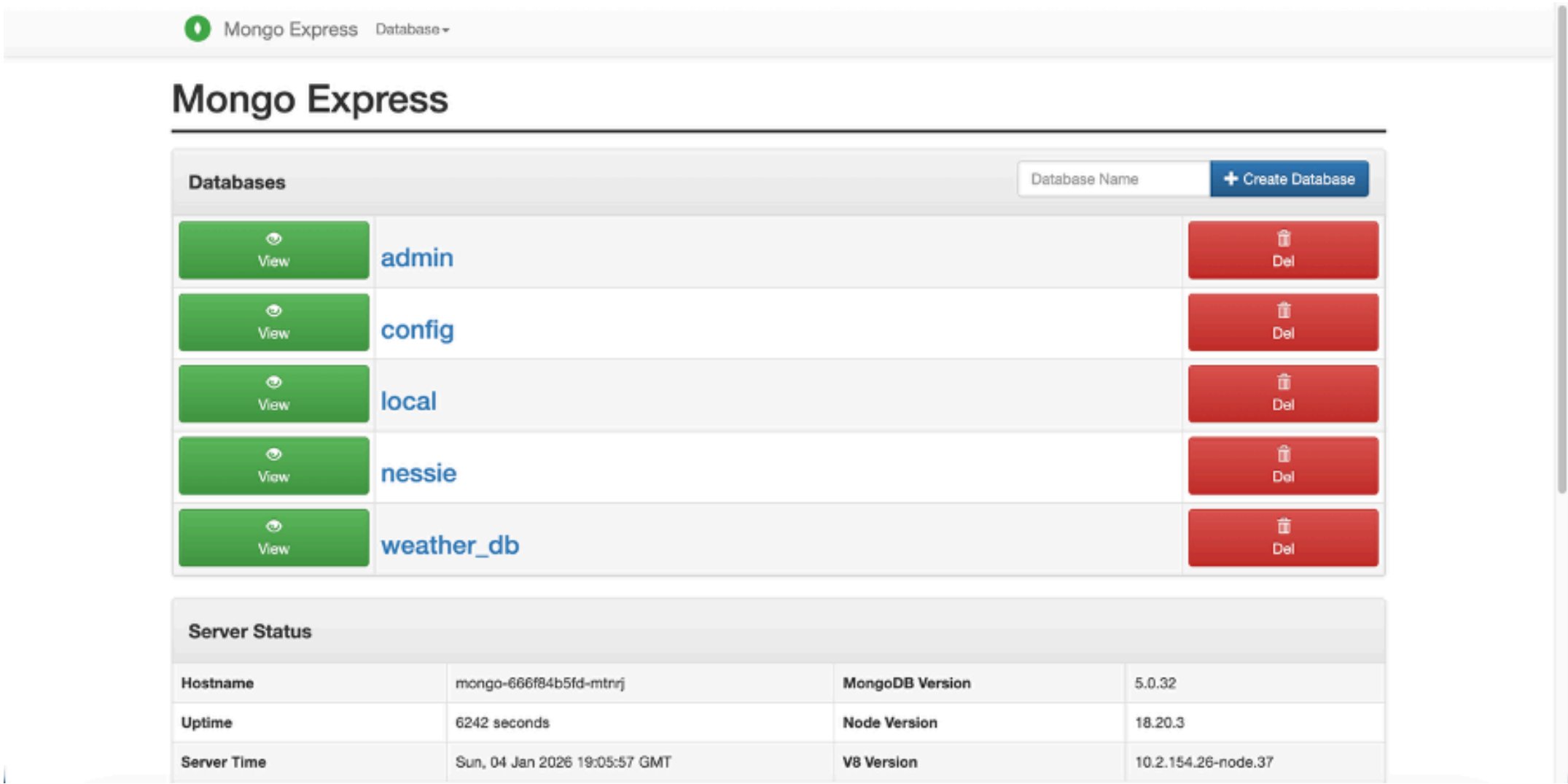
$$AT = T_{air} + 0.33 \times e_a - 0.70 \times V_{wind} - 4$$

4.3.2. Speed Layer

- A streamlined version of the **Bronze-to-Silver Transformation** in the **Batch layer**
- Omits **Wind Direction Vector Decomposition** and **WMA imputation with Fallback**
- **Stage 1: Stateless transformation**
 - Type conversion
 - String standardization
 - Outlier Nullification
- **Stage 2: Deduplication with Watermark**
 - Defines a watermark of **5 minutes** on the **ingested_at** timestamp column
 - Instructs Spark to maintain duplication state only for data arriving in this time window

4.4. Serving Database

- MongoDB
- Stores real-time views and preprocessed gold-tier batch views
- Also used by Nessie for version store
- 1GB PVC to store BSON documents
- Mongo Express as web UI for debugging and manual data validation



4.5. Visualization



Prometheus

Node Exporter
(system metrics)

Weather Exporter
(weather data)



Grafana

Grafana

Prometheus Alerts Graph Status Help

Targets

All scrape pools Unhealthy Collapse All

Filter by endpoint or labels

mongodb_exporter (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration
http://mongodb-exporter:9216/metrics	UP	instance="mongodb-exporter:9216" job="mongodb_exporter"	10.673s ago	5.26s

node_exporter (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration
http://node-exporter:9100/metrics	UP	instance="node-exporter:9100" job="node_exporter"	667.000ms ago	38.464ms

prometheus (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus"	13.468s ago	30.998ms

weather_exporter (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration
http://weather-exporter:8000/metrics	UP	instance="weather-exporter:8000" job="weather_exporter"	5.65s ago	36.400ms

- mongodb_exporter: Successfully capturing MongoDB database performance and storage metrics via port 9216.
- node_exporter: Monitoring hardware and OS-level metrics (CPU, Memory, Disk) on the cluster node via port 9100.
- weather_exporter: The custom application exporter is active and providing weather data ingestion metrics via port 8000.
- prometheus: The Prometheus instance is healthy for metric ingestion.

Grafana

Prometheus Alerts Graph Status Help

Targets

All scrape pools All Unhealthy Collapse All

Filter by endpoint or labels

mongodb_exporter (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration
http://mongodb-exporter:9216/metrics	UP	instance="mongodb-exporter:9216" job="mongodb_exporter"	10.673s ago	5.26s

node_exporter (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration
http://node-exporter:9100/metrics	UP	instance="node-exporter:9100" job="node_exporter"	667.000ms ago	38.464ms

prometheus (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus"	13.468s ago	30.998ms

weather_exporter (1/1 up) show less

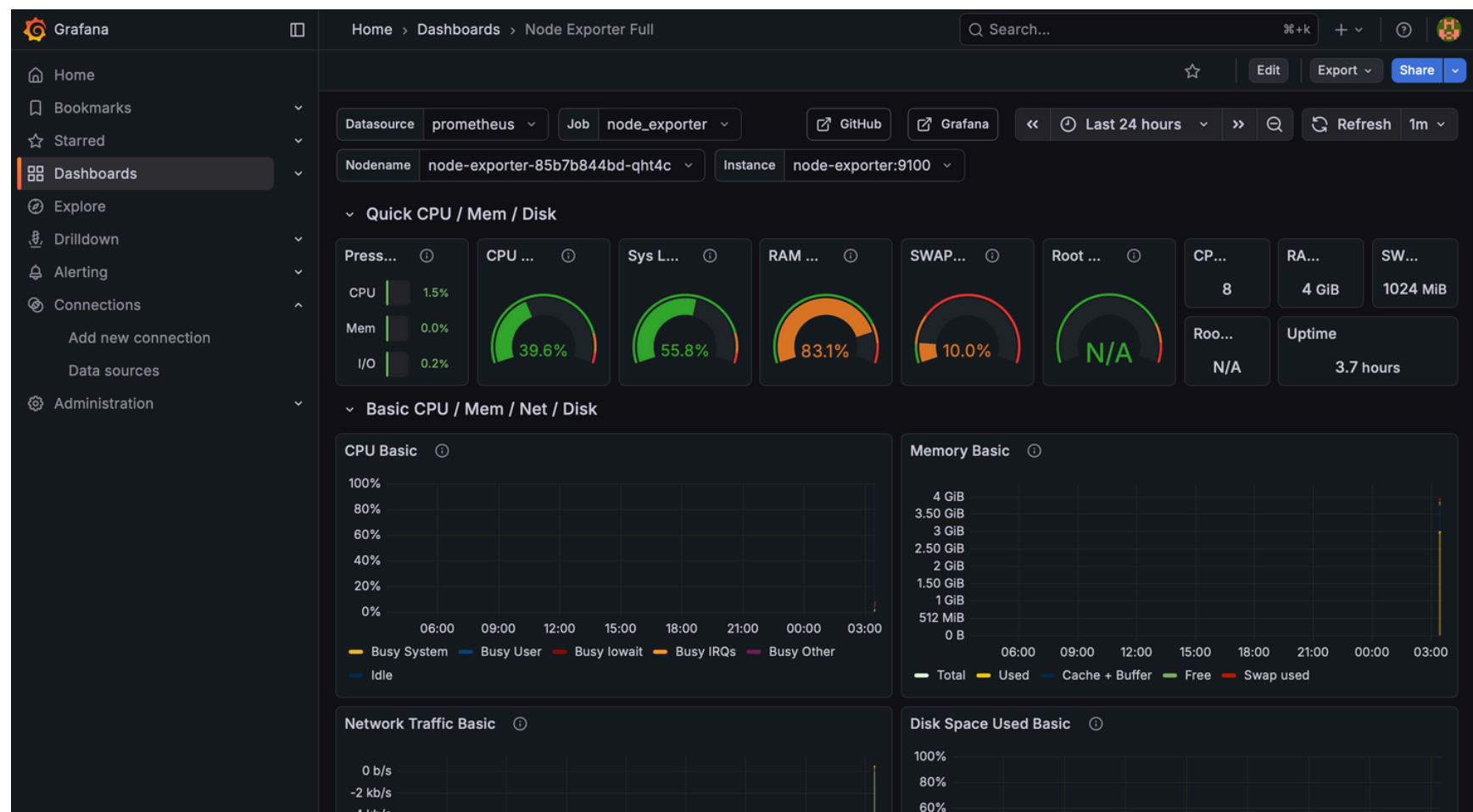
Endpoint	State	Labels	Last Scrape	Scrape Duration
http://weather-exporter:8000/metrics	UP	instance="weather-exporter:8000" job="weather_exporter"	5.65s ago	36.400ms

Weather Exporter

- A custom Python-based exporter
- Ingest the Gold Layer of the data pipeline, which contains highly processed, scientifically enriched meteorological features.
- Validated features from the Gold schema are mapped to Prometheus Gauge metrics

4.5. Visualization

Grafana



- Node exporter monitors the health of the host environment.
- Collects hardware and OS-level metrics, including CPU utilization, memory pressure, disk I/O throughput, and network traffic.
- Monitors and ensures that the computational resources running the ETL pipelines are operating within optimal parameters.

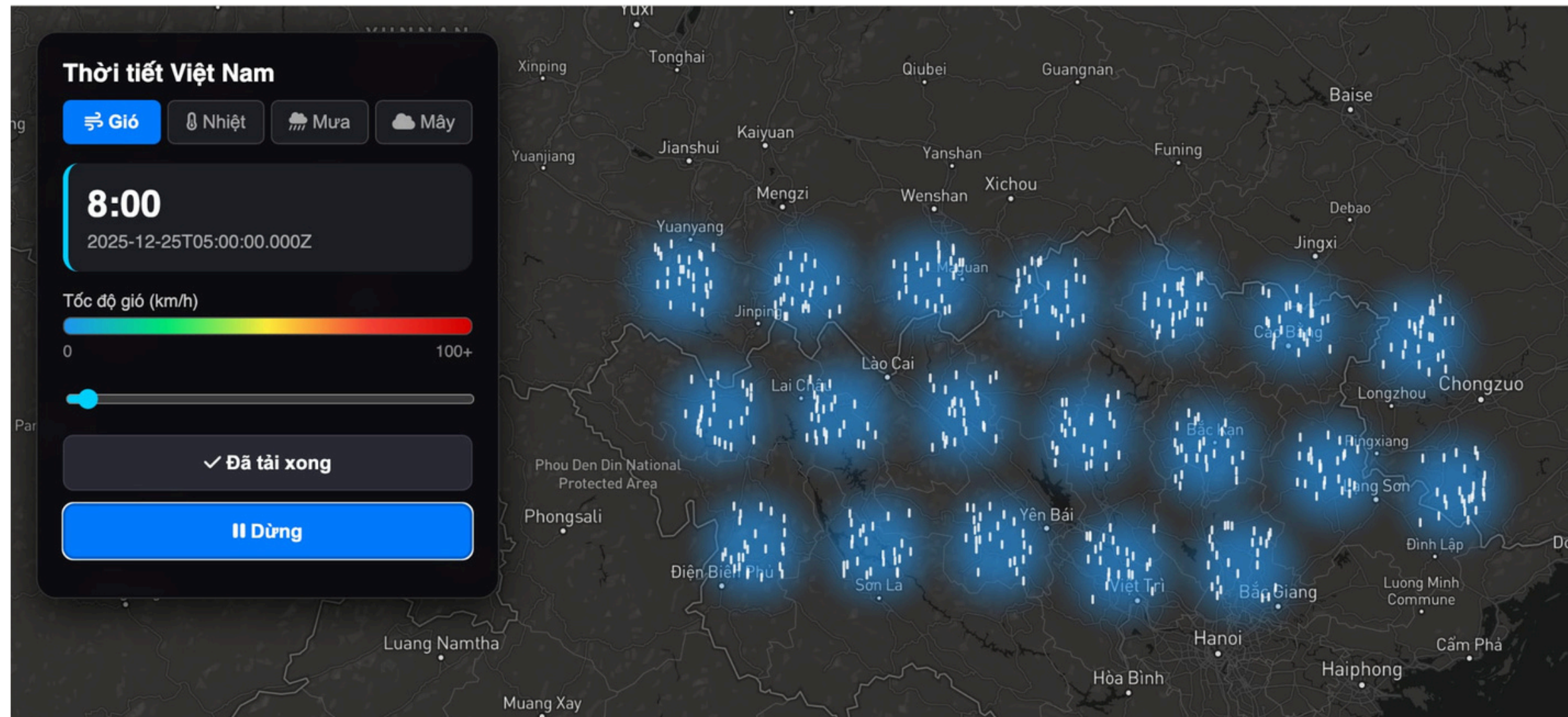
Mapbox

```
mapbox/
├── Dockerfile           # Node.js environment definition
├── backend/             # Server-side Logic
│   ├── server.js        # Entry point, API routing, and dynamic env injection
│   ├── dataLoader.js    # Data retrieval, transformation, and database interaction
│   ├── models.js        # MongoDB Schema definitions and Object Mapping
│   └── utils.js         # Helper functions (date processing, logic utilities)
└── frontend/           # Client-side Visualization
    ├── index.html       # Main entry for the web interface
    ├── css/             # Stylesheets for UI components
    └── js/
        ├── main.js      # Mapbox initialization and event listeners
        ├── api.js        # Asynchronous data fetching from Backend
        └── config.js     # Visualization parameters and color scales
```

Directory Structure

4.5. Visualization

Mapbox



Map Visualization

5. Lessons Learned

5. Lessons Learned

- Spark Processing Optimization:
 - Resolve OOM issue when running Spark jobs
 - Solved by separating Medallion batch jobs, caching, and eliminating Python loops and slow UDFs
- Version conflicts during development
 - Team members used different component versions during local development
 - Failure occurred during integration: Scala, Spark, and MongoDB
 - Solved by correcting versions; prevented by keeping track of images' versions
- Database Connection
 - Continuous failed authentications to MongoDB from Nessie and Spark
 - Tried providing credentials in different places: Kubernetes Secrets, hard-code, URI
 - Issue was due to how Mongo checks for user credentials; resolved by adding **?authSource=admin**
 - → Different DBs handle URIs and authentication differently; check out official docs

5. Lessons Learned

- Iceberg Catalog selection:
 - 3 options: Hadoop, REST, Nessie
 - Selected Nessie due to its benefits compared to Hadoop, whereas REST was difficult to configure
- Using Helm charts
 - During deployment of Nessie on K8s → failed authentication to MongoDB
 - Tried out hard-coding credentials to attributes and into database URI
 - Eventually resort to using simple Deployment + Service
 - → Helm charts could be hard to configure, and using Deployment is closer to configuring Docker containers

6. Conclusion

6. Conclusion

- Successfully implemented a **Lambda Architecture pipeline** on Kubernetes for real-time and batch processing
- **Built an end-to-end big data stack** integrating Kafka, Spark, MinIO, Apache Iceberg, and Project Nessie
- **Applied the Medallion Architecture** to transform raw weather telemetry into high-quality, analysis-ready datasets
- **Optimize streaming performance and scalability** to handle higher data velocity and larger deployments
- **Add data quality checks and anomaly detection** for more reliable weather forecasting

Member Contributions

Member Name	Contributions
Nguyễn Trí Thanh	- Researched on the topic for insights on data format and semantics - Designed the Mapbox app and Grafana to visualize weather history over time
Phan Trần Việt Bách	- Researched on the technologies used for the system - Designed and implemented the data ingestion stage including Kafka and Kafka Connect, the Nessie catalog, and worked on deploying them on Kubernetes
Nguyễn Minh Quân	- Designed, implemented and tested the Spark cluster used for data processing - Worked on deploying and integrating the system on Kubernetes
Lê Nhật Quang	- Researched on and implemented the transformation operations for the weather data using Spark - Contributed to implementing and testing the Kafka cluster
Nguyễn Hoàng Sơn	- Implemented and tested storage components, including MinIO and MongoDB - Implemented and tested connections from Spark to data storage



HUST

THANK YOU !



hust.edu.vn



fb.com/dhbkhn